

DOCUMENTO TÉCNICO CONSOLIDADO

Plan de Mejoramiento — Competencia Seguridad Informática

Sistema evaluado: SecureDesk ADSO — API de Gestión de Incidentes de Seguridad

Aprendiz: Manuela Córdoba Robledo

Identificación: 1015071897

Ficha: 3229209 · Programa: Tecnología en Análisis y Desarrollo de Software (ADSO) — SENA

Competencia: Seguridad Informática — RA 01

Instructor: Gustavo Bolaños

Versión del documento: 1.0

Fecha: 21 de septiembre de 2026

Repositorio: github.com/robledocordoba123-cmyk/securedesk-adso

Video YouTube: [PENDIENTE — completar antes de entregar]

Fecha de sustentación: [PENDIENTE — completar antes de entregar]

Tabla de contenido

Tabla de contenido	2
Introducción y nota metodológica.....	4
1. Descripción del sistema	4
2. Alcance	4
2.1 Incluido en esta fase	4
2.2 Fuera de alcance	5
3. Arquitectura	5
3.1 Endpoints principales.....	5
4. Activos.....	5
5. Amenazas y vulnerabilidades.....	6
6. Matriz de riesgos.....	6
7. Principios de confidencialidad, integridad y disponibilidad	7
7.1 Confidencialidad	7
7.2 Integridad.....	7
7.3 Disponibilidad	7
8. Autenticación	7
8.1 Evidencia — acceso sin token	7
8.2 Evidencia — login válido	7
9. Autorización (RBAC)	8
9.1 Capturas — permisos visibles según rol	8
10. JWT y Bcrypt.....	9
11. Gestión de secretos.....	9
12. Controles preventivos, detectivos y correctivos.....	10
13. Gestión de usuarios	10
13.1 Evidencia — desactivación y bloqueo de acceso	10
14. Backups	10
14.1 Evidencia real de ejecución.....	11
15. Restauración	11
15.1 Evidencia — antes del daño	11
15.2 Evidencia — simulación de pérdida	11
15.3 Evidencia — después de restaurar	11
16. Continuidad del servicio.....	12
17. RTO / RPO.....	12

18. Reversa (rollback).....	13
19. Pruebas de seguridad ejecutadas	13
20. Checklist de verificación preproducción	14
21. Hallazgos	14
22. Correcciones planificadas	14
23. Conclusiones	15
24. Referencias.....	15
24.1 Entregables propios del curso consolidados en este documento	15
24.2 Tecnologías y herramientas	15
24.3 Enlaces del plan de mejoramiento.....	15

Introducción y nota metodológica

Este documento consolida en un solo entregable el trabajo realizado durante las clases 1 a 8 de la competencia Seguridad Informática (RA 01 — Planear actividades de implantación del software), como parte del plan de mejoramiento asignado tras obtener 69.2/100 en el cierre del trimestre (mínimo aprobatorio: 70/100).

Nota metodológica: las clases 1, 2 y 6 de este curso se documentaron trabajando sobre RitmoApp, mi proyecto de grado (gestión de academias de baile), porque en ese momento era el sistema sobre el que estábamos definiendo arquitectura y roles con mi equipo. Sin embargo, la implementación técnica real — un backend en ejecución, con base de datos, pruebas ejecutadas y evidencia reproducible — se construyó sobre SecureDesk ADSO, la práctica integradora de referencia de la competencia (Prácticas 1 a 6, verificadas en la Clase 7 y consolidadas en el Plan de Implantación Segura de la Clase 8).

Este documento mantiene esa misma honestidad metodológica: cuando una sección proviene del marco conceptual trabajado sobre RitmoApp (por ejemplo, la estructura de un diagnóstico de riesgos o de un plan de continuidad), se indica explícitamente. Cuando una sección presenta evidencia técnica — pruebas ejecutadas, capturas, salidas de terminal —, esa evidencia es siempre de SecureDesk ADSO, el único sistema de este curso que tengo corriendo de verdad y que puedo auditar y defender en la sustentación.

1. Descripción del sistema

Campo	Descripción
Nombre oficial	SecureDesk ADSO
Versión	1.0 (ambiente de práctica/desarrollo). Aún no tiene versión de producción.
Propósito principal	API para la gestión de incidentes de seguridad: permite registrar, listar y clasificar incidentes (por severidad y estado), con control de acceso diferenciado según el rol del usuario.
Módulos activos	Autenticación (JWT), control de acceso por rol (RBAC), gestión de incidentes (con enmascaramiento de datos por rol), gestión de usuarios (activar/desactivar), respaldo y restauración de base de datos, registro de auditoría (logs Morgan).
Tipo de datos que maneja	Credenciales de usuario (correo y contraseña cifrada con bcrypt), rol asignado, e incidentes de seguridad con título, descripción, severidad, estado, correo del reportero (dato personal, enmascarado según rol) y fecha de creación.
Usuarios y roles	ADMIN (acceso total: ve datos completos, administra usuarios), ANALISTA (gestiona incidentes con datos personales enmascarados) y CONSULTA (solo lectura, con datos personales enmascarados).
Criticidad del sistema	Alta. Aunque es un proyecto académico, maneja credenciales, control de acceso y datos personales (correo del reportero): los mismos activos que comprometería un incidente real de seguridad.
Entorno de despliegue actual	Local / práctica: backend Node.js-Express en http://localhost:4001 , base de datos PostgreSQL en contenedor Docker "securedesk_db" (puerto 5433), frontend HTML/CSS/JS servido en el puerto 5500. Sin ambiente de producción desplegado todavía.

2. Alcance

2.1 Incluido en esta fase

- El backend completo de la API (rutas de autenticación, administración, usuarios e incidentes).
- La base de datos PostgreSQL con su esquema actual (User, Incident) y las migraciones ya aplicadas.
- El frontend HTML/CSS/JS que consume la API.

- Los tres roles de usuario ya definidos: ADMIN, ANALISTA y CONSULTA, con sus permisos actuales.
- Los scripts de respaldo y restauración (backend/scripts/backup.sh y restore.sh) y el flujo de gestión de usuarios (activar/desactivar).
- El repositorio del proyecto, ya publicado en GitHub con historial de commits y rama plan-mejoramiento.

2.2 Fuera de alcance

- Certificado TLS/HTTPS de producción y su renovación automática (hallazgo abierto, ver sección 22).
- Pruebas de carga con herramientas especializadas (k6, autocannon) contra concurrencia alta — solo se probaron peticiones individuales y ráfagas cortas.
- Nuevos módulos funcionales (notificaciones automáticas, reportes exportables, integración con un SIEM externo).
- Capacitación formal a usuarios reales — en esta fase los tres usuarios son cuentas de práctica creadas por mí.

3. Arquitectura

SecureDesk ADSO está organizado en tres capas separadas:

- Backend: Node.js + Express, organizado en rutas (auth, admin, users, incidents, health), middlewares (authenticateJWT, authorizeRoles, errorHandler) y validadores Zod.
- Base de datos: PostgreSQL administrada con Prisma ORM (modelos User e Incident, migraciones versionadas en backend/prisma/migrations).
- Frontend: HTML/CSS/JavaScript, separado del backend, consume la API por HTTP y adapta lo que muestra según el rol autenticado.

Seguridad aplicativa: JWT para sesión, bcrypt para contraseñas, express-rate-limit en el login, Morgan para trazabilidad, manejo centralizado de errores.

3.1 Endpoints principales

Endpoint	Roles permitidos	Función
GET /health	Público	Verifica que el servicio está activo
POST /auth/login	Público (con rate limit)	Autenticación, emite JWT
GET /admin/ping	ADMIN	Verifica acceso administrativo
GET /users	ADMIN	Lista usuarios
PATCH /users/:id/activate	ADMIN	Reactiva una cuenta
PATCH /users/:id/deactivate	ADMIN	Desactiva una cuenta
POST /incidents	ADMIN, ANALISTA	Crea un incidente (validado con Zod)
GET /incidents	ADMIN, ANALISTA, CONSULTA	Lista incidentes (correo enmascarado salvo ADMIN)

4. Activos

Identificados desde la Práctica 1 del Dossier de Evidencias, antes de programar el resto del sistema:

- Código fuente del backend y frontend (repositorio del proyecto).

- Credenciales y secretos: JWT_SECRET y credenciales de la base de datos, aislados en .env (nunca en el código ni en el repositorio).
- Variables de entorno por ambiente (.env.example como plantilla pública sin valores reales).
- Datos: base de datos PostgreSQL "securedesk" en un contenedor Docker dedicado.
- Logs de acceso y auditoría (generados por Morgan).
- Infraestructura: contenedor PostgreSQL y puertos de servicio (4001 API, 5433 PostgreSQL, 5500 frontend).

Pensar en estos activos desde el primer paso permitió decidir, antes de escribir el resto del sistema, qué información nunca debe salir del servidor (secretos) y qué sí puede exponerse públicamente (plantilla .env.example, endpoint de salud sin datos sensibles).

5. Amenazas y vulnerabilidades

Análisis realizado en la Práctica 2 del Dossier de Evidencias, antes de programar los controles, para priorizar qué construir primero:

- JWT_SECRET: amenaza de robo/filtración del secreto de firma; vulnerabilidad — secreto hardcodeado o subido al repositorio.
- Rutas /admin y /users: amenaza de escalamiento de privilegios; vulnerabilidad — falta de validación de rol en el backend (confiar solo en el frontend).
- Endpoint /auth/login: amenaza de ataque de fuerza bruta; vulnerabilidad — ausencia de límite de intentos.
- Datos recibidos en /incidents: amenaza de inyección de datos malformados o payloads maliciosos; vulnerabilidad — falta de validación/sanitización de entradas.
- Correo del reportero en Incident: amenaza de exposición de datos personales a roles sin necesidad de verlos; vulnerabilidad — falta de enmascaramiento por rol.
- Base de datos PostgreSQL: amenaza de pérdida o corrupción de datos; vulnerabilidad — ausencia de copias de respaldo verificadas.
- Cuentas de usuario: amenaza de uso continuo de una cuenta comprometida o de un ex-usuario; vulnerabilidad — falta de mecanismo para desactivar accesos.

6. Matriz de riesgos

Activo	Amenaza	Impacto	Prob.	Nivel	Control propuesto
JWT_SECRET	Robo/filtración del secreto	Alto — falsificación de tokens	Media	Alto	Solo en .env, nunca en código; .env excluido del control de versiones
/admin y /users	Escalamiento de privilegios	Alto — CONSULTA actuando como ADMIN	Media	Alto	Middleware authorizeRoles() obligatorio en cada ruta administrativa
/auth/login	Fuerza bruta	Alto — puerta de entrada al sistema	Alta	Crítico	express-rate-limit (5 intentos/minuto)
/incidents (entrada)	Inyección de datos malformados	Medio — datos corruptos o scripts almacenados	Media	Medio	Validación Zod + sanitización de strings antes de persistir
Correo del reportero	Exposición de datos personales	Medio — expone info. innecesariamente	Alta	Alto	Enmascaramiento de reporterEmail para roles distintos de ADMIN

Base de datos PostgreSQL	Pérdida/corrupción de datos	Alto — pérdida total de registros	Baja	Alto	Backups con pg_dump + restauración real probada con psql
Cuentas de usuario	Uso de cuenta comprometida/ex-usuario	Medio — acceso indebido sostenido	Media	Medio	Endpoints activate/deactivate + verificación de isActive en login

El riesgo priorizado como más crítico fue la ausencia de límite de intentos en /auth/login, porque es la puerta de entrada a todos los demás activos: si un atacante puede probar contraseñas sin restricción, cualquier otro control (RBAC, enmascaramiento, validación) pierde valor si logra entrar como ADMIN. Por eso ese control (Práctica 4) se priorizó justo después de RBAC.

7. Principios de confidencialidad, integridad y disponibilidad

7.1 Confidencialidad

Se protege mediante: contraseñas cifradas con bcrypt (nunca en texto plano, ni siquiera en los backups), JWT_SECRET separado del código en variables de entorno, y enmascaramiento del correo del reportero para roles distintos de ADMIN. Queda pendiente el cifrado en tránsito (HTTPS), documentado como Hallazgo 2 en la sección 22.

7.2 Integridad

Se protege mediante: validación estricta de entradas con Zod antes de persistir cualquier dato, sanitización de strings previa a la validación, y control de acceso por rol (RBAC) que impide que un usuario sin permisos modifique datos que no le corresponden. La integridad de los backups se verificó de forma activa: no basta con generar el archivo .sql, se comprobó que su restauración devuelve exactamente los mismos datos que existían antes del respaldo.

7.3 Disponibilidad

Se protege mediante: el endpoint /health para monitorear que el servicio está activo, rate limiting que evita que un ataque de fuerza bruta sature el endpoint de login, manejo centralizado de errores que evita caídas no controladas del servidor, y el procedimiento de backup/restauración probado de extremo a extremo, que permite recuperar el servicio ante una pérdida de datos en minutos y no en horas.

8. Autenticación

La autenticación se implementa con JSON Web Tokens (JWT). El endpoint POST /auth/login (backend/src/routes/auth.routes.js) verifica las credenciales contra la base de datos (contraseña comparada con bcrypt) y, si son válidas, emite un token firmado que incluye el id, el correo y el rol del usuario, con una expiración de 2 horas (JWT_EXPIRES_IN=2h). El middleware authenticateJWT (backend/src/middlewares/auth.js) decodifica y verifica ese token en cada ruta protegida.

8.1 Evidencia — acceso sin token

```
HTTP/1.1 401 Unauthorized
{"error":"Token no proporcionado"}
```

Resultado: el middleware authenticateJWT bloquea cualquier petición sin token antes de llegar al controlador — demuestra que la autenticación es obligatoria, no opcional, en las rutas protegidas.

8.2 Evidencia — login válido

Salida real de login (token truncado por longitud, el claim "role" es lo que valida authorizeRoles() en cada ruta):

```
{ "token": "eyJhbGciOiJIUzI1NiIs...",  
  "user": { "id": 1, "email": "admin@securedesk.local", "role": "ADMIN" } }
```

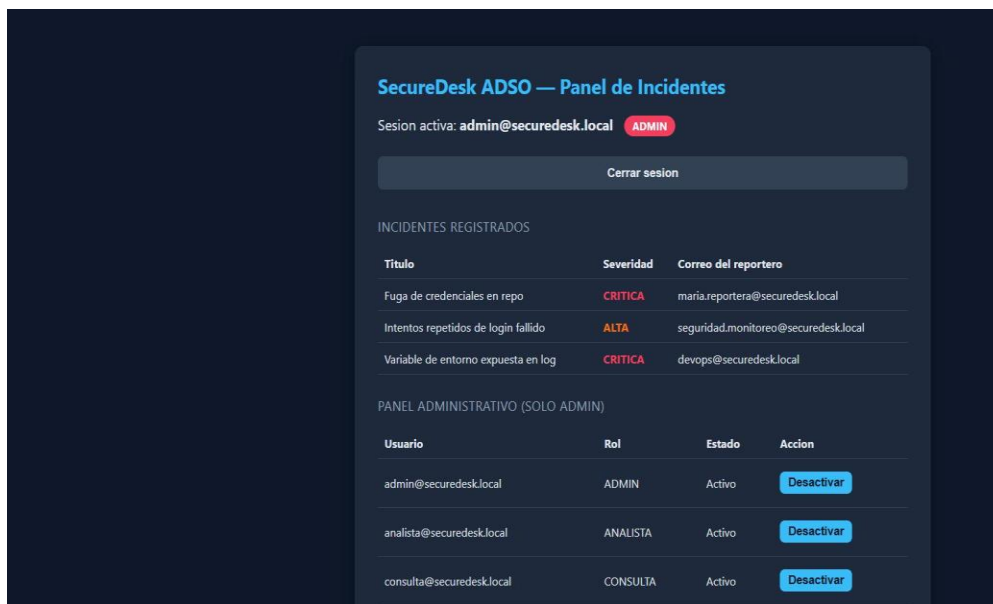
9. Autorización (RBAC)

El control de acceso por rol se implementa con el middleware authorizeRoles(...roles) (backend/src/middlewares/authorizeRoles.js), que se aplica después de authenticateJWT y compara el rol del token contra los roles permitidos para esa ruta. Se probaron los tres casos posibles sobre la misma ruta administrativa GET /admin/ping:

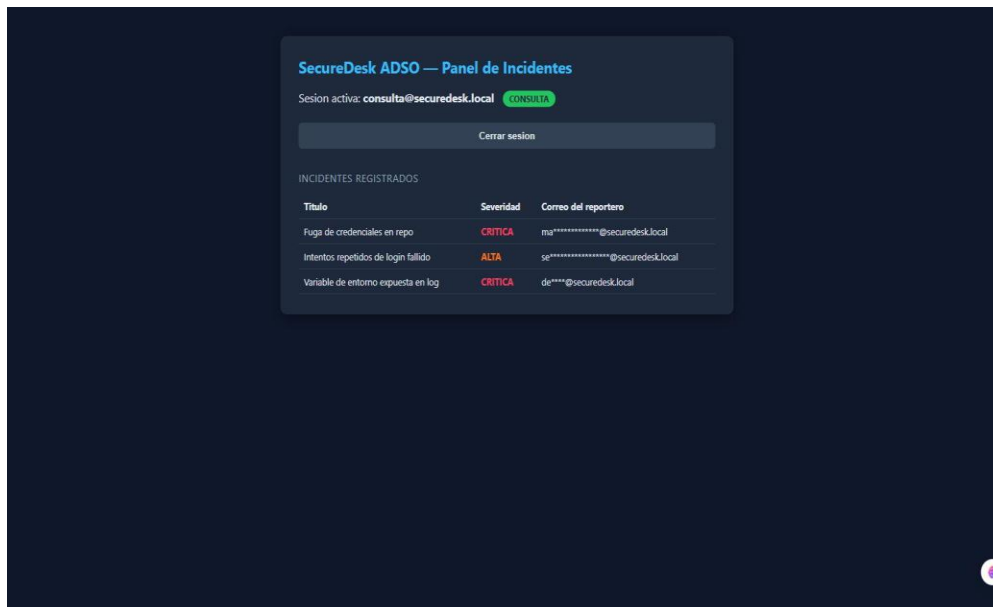
Caso	Resultado HTTP	Respuesta
Sin token	401 Unauthorized	{ "error": "Token no proporcionado" }
Token válido, rol CONSULTA (no autorizado)	403 Forbidden	{ "error": "No tiene permisos...", "rolRequerido": "ADMIN", "rolActual": "CONSULTA" }
Token válido, rol ADMIN (autorizado)	200 OK	{ "mensaje": "Acceso administrativo concedido", "usuario": "admin@securedesk.local", "rol": "ADMIN" }

La prueba más importante de esta sección es la de 403: confirma que la autorización se valida en el backend y no solo ocultando botones en el frontend. Si un usuario CONSULTA llamara directamente a la API con curl o Postman, sin pasar por la interfaz, seguiría recibiendo 403 — que es justo el riesgo que la sección 5 identificó como crítico (RBAC solo en frontend).

9.1 Capturas — permisos visibles según rol



Sesión como ADMIN: se ve el badge de rol ADMIN, la lista completa de incidentes con el correo del reportero sin enmascarar, y el panel administrativo de usuarios con la acción de activar/desactivar.



Sesión como CONSULTA: el panel administrativo no se muestra en absoluto (el frontend oculta lo que el rol no puede usar) y el correo del reportero aparece enmascarado, coherente con la respuesta real de la API.

10. JWT y Bcrypt

JWT (jsonwebtoken): firma tokens HS256 con el JWT_SECRET del servidor; el payload incluye sub (id), email y role; expiran a las 2 horas. Ningún dato sensible (contraseña) viaja dentro del token, solo el rol necesario para autorizar.

Bcrypt (bcryptjs): las contraseñas se almacenan como hash con salt, nunca en texto plano. Verificación real en el contenido del backup generado

(backend/backups/securedesk_backup_20260920_171316.sql):

```
admin@securedesk.local $2a$10$TkdM.Y07TRtaDbDrGk0S90V9zt1sZN0iGyKbuswUERtrZNZai3IAS ADMIN
```

Incluso con acceso directo al backup de la base de datos, la contraseña no es recuperable en texto plano — solo el hash bcrypt, que es justo lo que se espera de un manejo correcto de credenciales.

11. Gestión de secretos

Todos los secretos (JWT_SECRET, cadena de conexión a PostgreSQL) viven exclusivamente en backend/.env, que está excluido del repositorio mediante .gitignore. En su lugar, el repositorio publica backend/.env.example, una plantilla sin valores reales:

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST:PORT/securedesk?schema=public"
JWT_SECRET="CHANGE_ME_TO_A_LONG_RANDOM_SECRET"
JWT_EXPIRES_IN="2h"
PORT=4000
NODE_ENV=production
```

Hallazgo abierto (ver sección 22): el valor real de JWT_SECRET usado en el ambiente de práctica es un secreto de desarrollo, no uno generado para producción. La corrección definida es rotarlo por un valor

aleatorio de al menos 32 bytes, generado con un generador criptográfico, y cargarlo como variable de entorno del proveedor de despliegue — nunca en el repositorio.

También se excluyeron del repositorio, por la misma buena práctica de gestión de secretos, dos archivos de evidencia que contenían JWT completos generados durante las pruebas (admin_token.txt y consulta_token.txt): aunque ya habían expirado, publicar tokens crudos en un repositorio público no es una práctica aceptable, incluso como evidencia académica.

12. Controles preventivos, detectivos y correctivos

Tipo	Control	Qué evita
Preventivo	Validación Zod de entradas	Datos malformados o payloads maliciosos llegando a la base de datos
Preventivo	express-rate-limit en /auth/login (5/min)	Ataques de fuerza bruta contra credenciales
Preventivo	authorizeRoles() (RBAC)	Escalamiento de privilegios / acceso no autorizado a rutas administrativas
Preventivo	bcrypt para contraseñas	Exposición de contraseñas en texto plano ante una fuga de la base de datos
Detectivo	Morgan (logs de acceso)	Falta de trazabilidad — permite reconstruir qué pasó ante un incidente
Correctivo	Manejo centralizado de errores	Fuga de stack traces / información interna en respuestas de error
Correctivo	Backup + restauración con pg_dump/psql	Pérdida irreversible de datos ante error humano o migración fallida
Correctivo	Plan de reversa (sección 20)	Que un despliegue fallido deje el sistema en un estado peor que antes

Evidencia de los controles preventivos y correctivos trabajando en conjunto: al enviar un incidente con datos inválidos (título corto, severidad fuera del enum, correo mal formado), Zod y el manejador centralizado de errores responden con el detalle exacto de cada campo, sin exponer nunca una traza interna del servidor:

```
{"error":"Datos invalidos","detalles":[{"campo":"title","mensaje":"El titulo debe tener al menos 5 caracteres"}], ...}
```

13. Gestión de usuarios

Los endpoints GET /users, PATCH /users/:id/activate y PATCH /users/:id/deactivate (solo ADMIN) permiten administrar el ciclo de vida de las cuentas. El login verifica además el campo isActive: una cuenta desactivada no puede iniciar sesión aunque conozca su contraseña.

13.1 Evidencia — desactivación y bloqueo de acceso

```
PATCH /users/3/deactivate -> 200 OK
{"id":3,"email":"consulta@securedesk.local","isActive":false}
POST /auth/login (usuario desactivado) -> 403 Forbidden {"error":"Usuario desactivado,
contacte al administrador"}
```

Desactivar una cuenta bloquea el acceso de inmediato, incluso si la persona conoce su contraseña — mitigación directa del riesgo de "uso continuo de una cuenta comprometida o de un ex-usuario" identificado en la sección 5.

14. Backups

Qué se respalda: la base de datos PostgreSQL completa del contenedor "securedesk_db" (tablas User e Incident, incluidas las contraseñas ya cifradas con bcrypt), mediante backend/scripts/backup.sh, que ejecuta pg_dump.

Cuándo: antes de cualquier despliegue o migración de esquema, y de forma programada una vez en staging (propuesto: diario, dado el bajo volumen de datos del sistema).

Dónde se almacena: en backend/backups/, fuera del control de versiones (excluido por .gitignore, igual que el .env real).

14.1 Evidencia real de ejecución

```
Generando backup de PostgreSQL (securedesk) -> backups/securedesk_backup_20260920_171316.sql  
-rw-r--r-- 1 Pc 197609 7.0K Sep 20 17:13 backups/securedesk_backup_20260920_171316.sql
```

El archivo generado tiene 7.0 KB y contiene las filas reales de las tablas Incident y User, verificado inspeccionando su contenido directamente.

15. Restauración

Se ejecutó el ciclo completo, no solo la generación del backup: se simuló una pérdida de datos borrando toda la tabla Incident, y se restauró desde el archivo generado en la sección 14 con backend/scripts/restore.sh.

15.1 Evidencia — antes del daño

```
id | title  
1 | Intentos repetidos de login fallido  
2 | Variable de entorno expuesta en log  
3 | Fuga de credenciales en repo  
(3 rows)
```

15.2 Evidencia — simulación de pérdida

```
DELETE 3 -- se borra toda la tabla Incident por error
```

15.3 Evidencia — después de restaurar

```
id | title  
1 | Intentos repetidos de login fallido  
2 | Variable de entorno expuesta en log  
3 | Fuga de credenciales en repo  
(3 rows) -- Restauracion completada.
```

id #	title A	description A	severity E	status E	reporterEmail A	reportedById #	reportedBy ()
1	Intentos repetidos de L.	Se detectaron 12 intentos	ALTA	ABIERTO	seguridad.monitoreo@sec...	2	User
2	Variable de entorno exp.	Un log de desarrollo im...	CRITICA	ABIERTO	devops@securedesk.local	1	User
3	Fuga de credenciales en...	Se encontro un .env con...	CRITICA	ABIERTO	maria.reportero@secured...	1	User

Prisma Studio, tras la restauración: los 3 incidentes originales (ids 1, 2 y 3) están de nuevo en la base de datos, confirmando que la restauración devolvió el sistema al estado previo al incidente simulado. El proceso completo tomó menos de un minuto.

Ejecutar el ciclo completo —no solo generar el backup, sino además borrar datos a propósito y restaurarlos— es lo único que demuestra que un plan de respaldo funciona de verdad. Un archivo .sql que nunca se ha usado para restaurar es una promesa, no una garantía.

16. Continuidad del servicio

SecureDesk ADSO es una herramienta de apoyo para registrar y consultar incidentes de seguridad, no un sistema del que dependa una operación en tiempo real; esto permite una estrategia de continuidad simple:

- Modo manual temporal: si el sistema falla, los incidentes que ocurran mientras tanto se registran en una plantilla de respaldo (hoja de cálculo) con los mismos campos del modelo Incident, para cargarlos una vez esté restablecido.
- Suspensión controlada: si el fallo requiere restaurar la base de datos, el servicio se detiene por completo mientras dura el proceso, en vez de dejarlo respondiendo con datos parciales o inconsistentes.
- Sistema alternativo: no aplica en esta fase — no existe un sistema espejo activo; queda como mejora futura una vez el sistema pase a un uso productivo real.

Marco de referencia conceptual (trabajado en la Clase 6 sobre RitmoApp): definir prioridad, RTO y RPO por proceso crítico es la forma correcta de decidir cuánto esfuerzo de continuidad dedicarle a cada módulo, en lugar de tratar todo el sistema como si tuviera la misma urgencia. La sección 17 aplica ese mismo criterio directamente sobre SecureDesk ADSO.

17. RTO / RPO

Aplicando a SecureDesk ADSO el mismo criterio de priorización trabajado conceptualmente en la Clase 6 (RTO: tiempo máximo aceptable para recuperar el proceso; RPO: cuánta información como máximo se puede permitir perder):

Proceso crítico	Prioridad	RTO objetivo	RPO objetivo	Estado real
Autenticación (login)	Alta	15 minutos	0 — ninguna cuenta perdida	No probado en incidente real; el login depende de la misma base de datos respaldada
Gestión de incidentes	Alta	30 minutos	24 horas (frecuencia de backup propuesta)	Restauración real ejecutada en menos de 1 minuto (sección 15) — supera el objetivo
Gestión de usuarios	Media	30 minutos	24 horas	Cubierto por el mismo backup de base de datos
Registro/logs (Morgan)	Baja	No aplica (no crítico para operar)	Aceptable perder logs desde el último reinicio	N/A

La única métrica ya medida con evidencia real es el RTO de restauración de base de datos: el ciclo completo de backup, borrado simulado y restauración tomó menos de un minuto, muy por debajo del objetivo de 30 minutos propuesto para gestión de incidentes. Las demás filas son objetivos de diseño para esta fase, no mediciones — quedan pendientes de validar con una prueba de caída real, ver sección 22 (Hallazgos).

18. Reversa (rollback)

Punto de control: antes de cada despliegue se genera un backup completo de la base de datos (sección 14). El repositorio del proyecto ya está publicado en GitHub (<https://github.com/robledocordoba123-cmyk/securedesk-adso>), con la rama plan-mejoramiento como punto de referencia de esta entrega — el siguiente paso operativo es etiquetar (tag) cada versión estable antes de un despliegue real.

Criterio de activación: se activa la reversa si, tras un despliegue, el sistema deja de responder, la autenticación o el control de acceso por rol fallan, o se detecta pérdida/corrupción de datos que no se puede corregir en menos de 30 minutos.

- 1. Detener cualquier despliegue adicional en curso.
- 2. Volver al código de la última versión estable etiquetada en el repositorio.
- 3. Si hay corrupción real de datos, restaurar la base de datos desde el backup del punto de control (sección 15).
- 4. Verificar que el sistema funcione con la versión anterior: /health responde 200, login y RBAC operan con normalidad.
- 5. Documentar la reversa: hora de inicio y fin, versión restaurada, causa y resultado.

Responsable: Manuela Córdoba Robledo, como única responsable técnica del sistema en este contexto académico.

19. Pruebas de seguridad ejecutadas

Prueba	Resultado esperado	Resultado obtenido
GET /health	200 OK	200 OK — servicio activo
/admin/ping sin token	401	401 — "Token no proporcionado"
/admin/ping con rol CONSULTA	403	403 — rol rechazado
/admin/ping con rol ADMIN	200	200 — acceso concedido
POST /incidentes con datos inválidos	400 (Zod)	400 con detalle por campo
Ruta inexistente	404	404 controlado, sin traza interna

6 intentos seguidos a /auth/login	429 desde el 4.º intento	429 confirmado desde el 4.º intento
Login con cuenta desactivada	403	403 — "Usuario desactivado"
Backup + borrado simulado + restauración	Datos íntegros tras restaurar	3/3 registros recuperados, verificado en consola y Prisma Studio

Todas las pruebas se ejecutaron contra el backend real en funcionamiento (<http://localhost:4001>) y el contenedor PostgreSQL "securedesk_db", nunca sobre un diseño teórico.

20. Checklist de verificación preproducción

Ítem de verificación	¿Aprobado?
Separación de variables de entorno (.env real vs .env.example) y de puertos por servicio	Aprobado
Autenticación JWT y RBAC verificados en rutas administrativas	Aprobado
Validación y sanitización de datos de entrada (Zod) antes de persistir	Aprobado
Limitación de tasa (rate limiting) activa en el login	Aprobado
Manejo centralizado de errores sin exposición de stack traces	Aprobado
Trazabilidad y logs de acceso activos (Morgan)	Aprobado
Backup de base de datos generado correctamente	Aprobado
Restauración de backup probada, con integridad de datos verificada	Aprobado
Gestión de usuarios: cuentas desactivadas pierden acceso de inmediato	Aprobado
JWT_SECRET con nivel de robustez apto para producción	No aprobado
Certificado SSL/TLS (HTTPS) configurado	No aprobado
Prueba de disponibilidad bajo carga real	No aprobado

9 de 12 ítems ya están aprobados con evidencia real y reproducible. Los 3 pendientes son exactamente los que se documentan como hallazgos abiertos en la sección siguiente.

21. Hallazgos

Hallazgo	Impacto	Acción correctiva
JWT_SECRET usado en práctica es un valor de desarrollo, no un secreto de producción	Alto — si es predecible, se podrían falsificar tokens de cualquier rol	Generar un secreto aleatorio de al menos 32 bytes con un generador criptográfico y cargarlo como variable de entorno del proveedor, nunca en el repositorio
No hay certificado TLS/HTTPS; todo el tráfico de prueba corre sobre HTTP en localhost	Alto — en producción, tokens y credenciales viajarían en texto plano	Configurar HTTPS mediante certificado gestionado del proveedor de despliegue (o proxy inverso con TLS) antes de exponer el servicio públicamente
No se ejecutó una prueba de carga real (solo peticiones individuales y ráfagas cortas)	Medio — no se conoce el comportamiento ante concurrencia alta	Ejecutar una prueba de carga con k6 o autocannon simulando usuarios concurrentes antes del despliegue definitivo

22. Correcciones planificadas

Fase	Fechas	Actividades
Preparación	21–23 sep 2026	Rotar JWT_SECRET por un valor aleatorio de producción; documentar el procedimiento de generación del secreto.
Despliegue (staging)	24–25 sep 2026	Desplegar backend y base de datos en un proveedor externo; configurar HTTPS/TLS antes de exponer la URL pública.
Pruebas en staging	26–27 sep 2026	Repetir las pruebas de la sección 19 contra el ambiente de staging (no solo localhost); ejecutar la prueba de carga pendiente (Hallazgo 3).
Cierre y aprobación	28 sep 2026	Revisar checklist de criterios de salida; firmar aprobación funcional; autorizar o posponer el paso a producción.

Los dos hallazgos de impacto alto (JWT_SECRET y HTTPS) son correcciones concretas de corto plazo, no rediseños del sistema, y deben quedar cerrados antes de que el sistema reciba tráfico real.

23. Conclusiones

Construir SecureDesk ADSO práctica por práctica, en vez de documentar controles hipotéticos sobre un sistema que no existe, dejó claro que la seguridad de una API no depende de un solo control grande, sino de varios controles pequeños que se apoyan entre sí: la autenticación no sirve de nada sin autorización por rol, la autorización no sirve de nada si los datos que llegan al backend no se validan, la validación no sirve de nada si no queda registrada, y ningún control anterior importa si un error humano borra la base de datos y no hay con qué restaurarla.

Con base en las pruebas ejecutadas (sección 19), SecureDesk ADSO cumple de forma sólida los controles de seguridad aplicativa que dependen del código: el control de acceso se valida en el backend y no solo en la interfaz, las entradas se validan y sanitizan antes de persistir, el sistema responde de forma controlada ante fuerza bruta y rutas inexistentes, y el ciclo de respaldo y restauración se probó de extremo a extremo con datos reales.

Sin embargo, el sistema no está listo para producción todavía: dos de los tres hallazgos son de impacto alto y afectan directamente la confidencialidad en producción (JWT_SECRET de desarrollo y ausencia de HTTPS). Mi decisión es postergar el paso a producción hasta cerrar esos dos hallazgos — son correcciones concretas y de corto plazo, no rediseños del sistema — mientras que la prueba de carga (impacto medio) puede resolverse en paralelo al primer despliegue en staging.

A nivel de plan de mejoramiento, este documento, junto con el repositorio publicado en GitHub (con código funcional, evidencia real y la rama plan-mejoramiento) y el video técnico y la sustentación pendientes, buscan demostrar no solo que el sistema funciona, sino que entiendo por qué cada control existe, qué riesgo mitiga, y qué falta por cerrar antes de confiar este sistema con datos reales.

24. Referencias

24.1 Entregables propios del curso consolidados en este documento

- Diagnóstico Inicial y Mapa Básico de Riesgos (Clase 1, sobre RitmoApp).
- Checklist de Implantación Segura (Clase 2, sobre RitmoApp).
- Análisis de Caso y Checklist de Aceptación Segura (Clase 5).
- Plan de Respaldo, Restauración, Continuidad y Contingencia (Clase 6, sobre RitmoApp).
- Dossier de Evidencias Prácticas, Clases 1 a 6 (SecureDesk ADSO).
- Informe de Verificación Preproducción, Clase 7 (SecureDesk ADSO).
- Plan de Implantación Segura, Clase 8 (SecureDesk ADSO).

24.2 Tecnologías y herramientas

- Node.js, Express, PostgreSQL, Prisma ORM.
- jsonwebtoken (JWT), bcryptjs, zod, express-rate-limit, morgan, cors.
- pg_dump / psql para backup y restauración.

24.3 Enlaces del plan de mejoramiento

Repositorio GitHub: <https://github.com/robledocordoba123-cmyk/securedesk-adso> (rama plan-mejoramiento)

Video YouTube: [PENDIENTE — completar antes de entregar]

Fecha de sustentación: [PENDIENTE — completar antes de entregar]